

Gemini AI developer handbook

A field guide for building useful, grounded, and responsible AI features with the Gemini API and Google GenAI SDK.

COURSE PROMISE

No AI background is required. Each capability starts with a plain-English mental model, then becomes a small working pattern, a safety check, and a team discussion.

Core track

20 hours / 10 chapters / server-side JavaScript / Google GenAI SDK / Interactions API / Gemini 3.5 Flash baseline / Team Knowledge Assistant capstone

Edition

Version 1.0 - Gemini developer edition. Official Google AI documentation reviewed 12 July 2026. Model names and preview features change; verify before production use.

Contents

The page numbers below are generated from the final rendered document.

00 / Start here	3
01 / What Gemini is - and is not	5
02 / Your first Gemini API call	7
03 / Prompts that teammates can maintain	9
04 / Conversations, streaming, and long context	11
05 / Images, documents, audio, and video	13
06 / Structured output and function calling	15
07 / Grounding, retrieval, and trustworthy answers	17
08 / Safety, security, cost, and production	19
09 / Capstone - Team Knowledge Assistant	21
Appendix A / Setup and troubleshooting	23
Appendix B / Review checklists and glossary	24

00 / Start here

In plain English

Learn the shape of the course, the project you will build, and the rules that keep experiments safe.

New words you will meet: model, prompt, token, interaction

This handbook is for developers, analysts, designers, and technical teammates who want to build a useful feature with the **Gemini API**. You do not need a machine-learning degree. You do need basic JavaScript, the command line, and a willingness to test what the model actually does.

The course follows one project: **Team Knowledge Assistant**. It answers questions from approved team material, returns a predictable response shape, and asks your application to perform a small tool action only after the user confirms it.

The learning loop

Every chapter uses the same four moves:

- Understand the idea in ordinary language.
- Run the smallest working example.
- Add a failure case and observe it.
- Write down the rule your team will follow.

The important boundary

Gemini proposes language, structure, or a tool request. Your application owns authentication, authorization, validation, side effects, logs, user experience, and the final decision to trust or reject an output.

What you will build

- A server-side Gemini client with a protected key.
- A reusable prompt template for team questions.
- A streamed answer experience.
- Image and document understanding.
- Schema-validated JSON for the user interface.
- A function-calling loop that never executes blindly.
- Grounded answers with source links.
- Safety, evaluation, cost, and production checklists.

Before you begin

Use synthetic or approved data while learning. Do not paste credentials, customer records, private source code, health information, or confidential company documents into an experiment unless your organization has explicitly approved the service, account, region, retention settings, and workflow.

First team decision

Write one sentence describing data that is allowed in the course project, and one sentence describing data that is forbidden. Put both sentences in the repository README.

01 / What Gemini is - and is not

In plain English

Gemini is a family of models. Your application sends input, receives output, and remains responsible for decisions.

New words you will meet: model, inference, multimodal, hallucination

Gemini is a **family of multimodal generative models**. “Multimodal” means a model may accept more than text, such as images, audio, video, or documents, depending on the chosen model and API feature.

Your program sends input to an API. The service runs a model and returns output. The output may be fluent and useful, but it is not a database lookup, a proof, or a promise of truth.

A simple mental model

Think of Gemini as a very capable draft partner:

- It can summarize, classify, extract, transform, compare, and plan.
- It can interpret mixed media and natural-language instructions.
- It can request a tool, but your code decides whether the tool runs.
- It can be wrong while sounding confident.
- The same prompt can produce different valid responses.

This is why an AI feature needs normal software engineering plus evaluation of model behavior.

Model, API, and product are different things

The **model** is the learned system that produces output. The **Gemini API** is the developer interface. **Google AI Studio** is a browser workspace for experimenting with prompts and keys. The consumer Gemini app is a finished product with its own interface and policies. This handbook focuses on the developer API.

Choosing a model without guessing

Model names and preview status change. Start from the official model table on the day you build. Compare capability, latency, input types, context limits, price, region, stability, and deprecation status.

For the examples in this edition, `gemini-3.5-flash` is the teaching baseline because the current official quickstart uses it. A team may choose a stable Flash model for routine high-volume work and a Pro model for harder reasoning, but that is a hypothesis to measure—not a universal rule.

Preview means change

A preview model or feature may change behavior, limits, or availability. Pin the model identifier, record the date you evaluated it, monitor deprecations, and plan a replacement test before adopting it in a critical workflow.

Good first use cases

- Turn meeting notes into action-item drafts.
- Classify support messages into a small approved taxonomy.
- Extract fields from a known document format.
- Answer questions from a controlled knowledge collection.
- Draft a response that a human reviews before sending.

Avoid making the first project an autonomous, irreversible, high-stakes decision maker.

02 / Your first Gemini API call

In plain English

Create a protected API key, install Google's current SDK, and make one small request from server-side JavaScript.

New words you will meet: API key, SDK, environment variable, response

The current official path for new projects is the **Interactions API** through the **Google GenAI SDK**. The original generateContent API remains supported, but this course uses Interactions because Google recommends it for new work and new capabilities.

Create a small server-side project

TERMINAL

```
mkdir gemini-team-assistant
cd gemini-team-assistant
npm init -y
npm install @google/genai
```

Add "type": "module" to package.json. Put the key in an environment variable, never in source code.

TERMINAL

```
export GEMINI_API_KEY="replace-with-your-key"
```

Create first-call.js:

JAVASCRIPT

```
import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({});

const interaction = await ai.interactions.create({
  model: "gemini-3.5-flash",
  input: "Explain an API in two beginner-friendly sentences.",
});

console.log(interaction.output_text);
```

Run it:

TERMINAL

```
node first-call.js
```

The SDK reads `GEMINI_API_KEY`. Your code creates an interaction, waits for the response, and prints the text output.

Keep the key on the server

A browser bundle, mobile application, public repository, screenshot, or copied terminal log is not a safe home for a long-lived key. Route browser requests through your authenticated backend. Restrict keys as Google recommends, separate development and production projects, rotate exposed credentials, and set usage alerts.

JAVASCRIPT

```
// server/gemini.js
import "dotenv/config";
import { GoogleGenAI } from "@google/genai";

if (!process.env.GEMINI_API_KEY) {
  throw new Error("GEMINI_API_KEY is missing");
}

export const ai = new GoogleGenAI({});
```

Handle failure as a normal path

Network calls fail. Rate limits happen. A request can be rejected or time out. Show the user a useful message and keep enough diagnostic information for developers without logging secrets or raw sensitive prompts.

JAVASCRIPT

```
export async function explainTerm(term) {
  try {
    const result = await ai.interactions.create({
      model: "gemini-3.5-flash",
      input: `Explain this term for a new developer: ${term}`,
    });
    return { ok: true, text: result.output_text };
  } catch (error) {
    console.error("Gemini request failed", { name: error.name });
    return { ok: false, text: "The assistant is unavailable. Try again." };
  }
}
```

Make failure visible

Run the first call successfully. Then temporarily use a bad key and disconnect the network. Record what the SDK throws and what the user should see in each case.

03 / Prompts that teammates can maintain

In plain English

Turn vague requests into clear instructions with context, constraints, examples, and a useful output shape.

New words you will meet: system instruction, context, few-shot, evaluation

A prompt is part instruction, part input data, and part interface contract. A useful prompt tells the model its task, supplies relevant context, defines constraints, and explains what a good answer looks like.

Start with a prompt contract

Before polishing words, write:

- Purpose: what job is being completed?
- Audience: who will use the answer?
- Inputs: what information is supplied?
- Output: what must be returned?
- Limits: what must the model refuse or avoid?
- Evidence: how will the team know it worked?

STRUCTURE

TASK

Summarize an internal project update for a new teammate.

CONTEXT

Use only the update between <project_update> tags.

RULES

- Do not invent owners, dates, or decisions.
- Mark missing information as "Not stated".
- Use plain English.

OUTPUT

Return: goal, progress, blockers, decisions, next actions.

Clear delimiters help distinguish instructions from untrusted content. They do not create a security boundary by themselves.

System instruction versus user input

Stable application behavior belongs in a system instruction. The current request and user-provided material belong in the input. Keep authorization rules in code; do not rely on a system instruction to protect data or approve actions.

JAVASCRIPT

```
const interaction = await ai.interactions.create({
  model: "gemini-3.5-flash",
  system_instruction: [
    { type: "text", text: "You explain company terms for new hires." },
    { type: "text", text: "If context is missing, say what is missing." },
  ],
  input: "Explain the term: release train",
});
```

Examples teach format

One or two compact examples can clarify labels, tone, or edge cases. Do not bury the real task beneath dozens of examples. Keep examples representative and test whether they accidentally bias the answer.

Prompts are versioned software artifacts

Store important prompts in named files or functions. Review changes, attach a version to logs, and run a small evaluation set before merging. “It looked good once in the playground” is not a test.

Create an evaluation table with ordinary cases, confusing cases, malicious instructions inside user content, missing context, very long input, and unsupported language. Give each case an expected property rather than one exact sentence.

Evaluate properties, not vibes

Useful checks include: required fields exist, claims are supported, forbidden data is absent, tone is acceptable, latency stays within budget, and a human can recover when the answer is uncertain.

04 / Conversations, streaming, and long context

In plain English

Carry a conversation forward, stream slow answers, and manage large inputs without confusing the model.

New words you will meet: interaction, state, stream, context window, cache

An Interaction represents one complete turn. The API can store interaction state and let a later turn reference `previous_interaction_id`, so your application does not have to resend the complete conversation each time.

JAVASCRIPT

```
const first = await ai.interactions.create({
  model: "gemini-3.5-flash",
  input: "My project is a knowledge assistant for new hires.",
});

const second = await ai.interactions.create({
  model: "gemini-3.5-flash",
  previous_interaction_id: first.id,
  input: "Suggest three acceptance tests for it.",
});
```

Server-side conversation state is convenient, but it is also a data-retention decision. The Interactions API stores requests by default for state management. Use `store: false` for stateless behavior when appropriate, and align the choice with your organization's data policy.

Streaming improves perceived speed

For a long answer, show text as it arrives. The application must still handle cancellation, partial output, network failure, and moderation. Never treat a partial JSON object as valid structured data.

JAVASCRIPT

```
const stream = await ai.interactions.create({
  model: "gemini-3.5-flash",
  input: "Create a beginner checklist for reviewing an AI feature.",
  stream: true,
});

for await (const event of stream) {
  if (event.event_type === "step.delta" && event.delta.type === "text") {
    process.stdout.write(event.delta.text);
  }
}
```

Long context is not perfect memory

A large context window lets the model consider a large input, but relevant facts can still be missed. Put stable instructions first, remove irrelevant material, label sections, and ask focused questions. Use retrieval when the knowledge collection is larger than one sensible request.

Context caching can reduce repeated processing cost and latency for a large shared prefix. Measure cache hits and confirm the minimum token and model requirements in the current documentation.

05 / Images, documents, audio, and video

In plain English

Gemini can understand more than text. You will send media safely and ask focused questions about it.

New words you will meet: multimodal, MIME type, file, grounding

Multimodal input lets a model reason about media together with text. The question still matters. “What is in this?” is weaker than “List the visible safety hazards and identify which observation supports each item.”

A reliable media workflow

- Confirm the model supports the input type.
- Validate file type and size before upload.
- Remove or mask data the model does not need.
- State the exact question and output format.
- Treat extracted text and descriptions as untrusted.
- Show the original source beside important conclusions.

Documents may contain embedded instructions such as “ignore the previous rules.” Those words are document content, not trusted application policy. This is one form of prompt injection.

Ask for observable evidence

For an image, request locations or visible features. For a document, request page references or quoted field labels. For audio, request timestamps when supported. For video, split broad questions into scenes, events, and evidence.

Human review for consequential interpretation

Do not use a general multimodal model as the sole authority for medical images, identity, legal conclusions, safety certification, or other high-impact judgments. Use domain-approved systems and qualified reviewers.

Practice exercise

Use a synthetic one-page project brief. Ask Gemini to extract the project owner, deadline, risks, and open questions. Then remove the deadline from the document. A good workflow should return “not stated,” not invent a plausible date.

06 / Structured output and function calling

In plain English

Use schemas when software needs dependable JSON, and tools when the model must request an action from your code.

New words you will meet: JSON Schema, validation, tool, function call

Use **structured output** when your program needs data in a predictable JSON shape. Use **function calling** when Gemini should choose or prepare a request to one of your application's tools.

Structured output

Describe the shape with JSON Schema, then validate the returned JSON in your application. A schema improves structure; it does not prove that the values are true.

JAVASCRIPT

```
const feedbackSchema = {
  type: "object",
  properties: {
    category: { type: "string", enum: ["bug", "idea", "question"] },
    summary: { type: "string" },
    needsHumanReview: { type: "boolean" },
  },
  required: ["category", "summary", "needsHumanReview"],
};

const result = await ai.interactions.create({
  model: "gemini-3.5-flash",
  input: "Classify: The export button downloads an empty file.",
  response_format: {
    type: "text",
    mime_type: "application/json",
    schema: feedbackSchema,
  },
});

const feedback = JSON.parse(result.output_text);
```

Production code should validate feedback with a schema library and reject extra, invalid, or oversized values.

Function calling

A function declaration tells the model which action exists and which arguments it accepts. The model may return a function-call step. Your code must validate the name and arguments, check the current user's permission, ask for confirmation when needed, execute the real function, and return an appropriate result.

JAVASCRIPT

```
const createDraftTask = {
  type: "function",
  name: "create_draft_task",
  description: "Creates a draft task. It does not publish or assign it.",
  parameters: {
    type: "object",
    properties: {
      title: { type: "string" },
      owner: { type: "string" },
    },
    required: ["title"],
  },
};

const result = await ai.interactions.create({
  model: "gemini-3.5-flash",
  input: "Draft a task to update the onboarding guide for Priya.",
  tools: [createDraftTask],
});

for (const step of result.steps) {
  if (step.type === "function_call") {
    console.log(step.name, step.arguments); // inspect; do not auto-run
  }
}
```

The tool boundary

The model suggests: “call this tool with these arguments.” Your application decides: “is this tool allowed, are the arguments valid, may this user do it, and should we ask for confirmation?”

Design safer tools

Prefer narrow tools such as `create_draft_task` over broad tools such as `run_database_query`. Separate read tools from write tools. Use allowlists, idempotency keys, timeouts, audit logs, and least-privilege credentials. Make irreversible actions a second explicit step.

07 / Grounding, retrieval, and trustworthy answers

In plain English

Connect answers to current or private sources and show users where claims came from.

New words you will meet: grounding, citation, File Search, RAG

Grounding connects an answer to external information. It can reduce unsupported claims and help a user inspect sources, but it does not make every claim correct.

Three common patterns

- **Google Search grounding** for current public information.
- **URL Context** for specific public pages supplied to the model.
- **File Search** for retrieval-augmented generation over an indexed collection.

File Search imports, chunks, and indexes your material, retrieves relevant passages, and provides them as context. This is useful for handbooks, policies, and product documentation that are too large to place in every prompt.

Build an answer users can inspect

A grounded assistant should distinguish sourced facts from model explanation, preserve citation metadata, link to the actual source, show “I could not find this” when retrieval is weak, and avoid citing a source that does not support the nearby claim.

Retrieval quality is a system problem

Poor answers may come from bad source files, weak chunking, missing metadata, vague queries, or a prompt that encourages guessing. Evaluate retrieval separately from generation:

- Did the system retrieve the correct passage?
- Was the passage current and authorized for this user?
- Did the answer use it faithfully?
- Is the citation attached to the supported claim?

Create an unanswerable question

Add five approved documents to a test collection. Ask four answerable questions and one question whose answer is absent. The assistant should cite the first four and clearly decline to invent the fifth.

08 / Safety, security, cost, and production

In plain English

Protect keys and user data, test risky behavior, watch usage, and design graceful failure paths.

New words you will meet: safety filter, prompt injection, rate limit, observability

An AI feature processes untrusted natural language and produces probabilistic output. Production readiness therefore needs more than a successful API call.

Threats to plan for

- A leaked API key causes unauthorized usage and cost.
- A user prompt tries to override application instructions.
- A retrieved document contains prompt injection.
- The model requests a tool the user may not execute.
- Sensitive data appears in prompts, output, logs, or analytics.
- A plausible answer causes a user to skip verification.
- Retries multiply cost or repeat a side effect.

Treat user input, retrieved text, URLs, media, model output, and tool arguments as untrusted data.

Safety filters are one layer

Gemini provides configurable safety settings for supported harm categories. Choose settings during risk assessment and test them with representative inputs. Also add product rules, access control, input limits, output validation, human review, and a reporting path. Do not weaken filters simply to make a demo pass.

Observe the system without collecting everything

Useful telemetry includes request ID, prompt-template version, model identifier, latency, token usage, cache usage, finish state, tool name, validation result, error category, and user feedback. Avoid raw sensitive content by default. Use redaction and short retention where diagnostics require samples.

Control cost and latency

- Select the smallest model that passes the evaluation.
- Limit output length and remove irrelevant context.

- Stream when perceived latency matters.
- Cache repeated large context where supported.
- Set per-user quotas and global budgets.
- Use bounded retries with backoff for transient failures.
- Never retry a write tool blindly.

A release gate

Before launch, confirm ownership, approved data classes, key restrictions, model/version choice, evaluation results, safety settings, authorization checks, fallback UI, rate limits, usage alerts, logs, incident response, deprecation monitoring, and a rollback switch.

09 / Capstone - Team Knowledge Assistant

In plain English

Combine prompting, documents, structured output, tools, safety, and evaluation into one useful internal assistant.

New words you will meet: capstone, acceptance test, rollout, feedback

The capstone answers questions from approved onboarding material and can prepare—but not publish—a follow-up task.

User story

“As a new teammate, I can ask a question about approved team processes, see a concise answer with sources, and create a draft follow-up task after reviewing it.”

Architecture in plain English

The browser sends an authenticated request to your server. The server validates it, retrieves only documents the user may access, asks Gemini for a grounded structured answer, validates the response, and returns safe fields to the UI. If Gemini requests `create_draft_task`, the server shows a confirmation screen before calling the task system.

Minimum answer contract

JSON

```
{
  "answer": "string",
  "confidenceNote": "string",
  "sources": [
    { "title": "string", "url": "string" }
  ],
  "followUpQuestion": "string"
}
```

Acceptance tests

- A common question returns a short answer and a supporting source.
- An absent answer is reported as missing instead of invented.
- A user cannot retrieve a document outside their access scope.

- Instructions inside a document cannot activate a tool.
- Invalid structured output is rejected and replaced by a safe retry or fallback.
- A task stays a draft until the user confirms it.
- Rate limiting and service failure produce usable messages.
- Logs identify the model and prompt version without storing prohibited data.

Suggested course demo

Show one successful answer, one unanswerable question, one blocked unauthorized document, one prompt-injection attempt, one schema-validation failure, and one confirmed draft task. A trustworthy failure demo is as important as the happy path.

Team retrospective

Which part of the result is deterministic software, which part is probabilistic model behavior, and where could a future teammate mistakenly trust the wrong layer?

Appendix A / Setup and troubleshooting

In plain English

Use this when installation, authentication, or an API request does not behave as expected.

New words you will meet: Node.js, npm, status code, retry

Quick setup checklist

- Use a currently supported Node.js version.
- Install @google/genai, not the legacy @google/generativeai package.
- Export GEMINI_API_KEY in the server environment.
- Keep .env files out of Git.
- Confirm the chosen model exists and supports the needed capability.
- Run a one-line text request before adding frameworks or tools.

Common failures

Authentication error: confirm the key is present, active, restricted correctly, and associated with the intended project.

Model not found: check the current model documentation, region, endpoint, model identifier, and deprecation page.

Rate limit: reduce concurrency, queue work, respect retry information, and request an appropriate quota or billing tier.

Blocked or empty output: inspect safety/finish metadata, shorten or clarify the request, and show the user a safe recovery path.

Slow response: measure network time, model time, input size, output size, tool calls, and retries separately. Consider streaming and a smaller model after evaluation.

Invalid JSON: use structured output, validate the schema, keep the schema simple, and retry only when safe and bounded.

Appendix B / Review checklists and glossary

In plain English

Use these lists during implementation and look up unfamiliar Gemini vocabulary quickly.

New words you will meet: review, glossary, source

Pull-request checklist

- The key remains server-side and is not logged.
- The user is authenticated and authorized before private retrieval or tools.
- Prompts have a name, version, purpose, and evaluation cases.
- Inputs have size limits and untrusted content is clearly separated.
- Structured output and tool arguments are validated.
- Side effects require least privilege and appropriate confirmation.
- Errors, timeouts, cancellation, and rate limits have user-facing paths.
- Model name, preview status, cost, and deprecation plan are recorded.
- Safety and prompt-injection tests are included.

Tiny glossary

Context: information available to the model for the current request.

Grounding: connecting an answer to external sources or retrieved evidence.

Hallucination: unsupported or incorrect output presented as if it were reliable.

Interaction: the Interactions API resource representing a complete turn and its execution steps.

Multimodal: able to work with more than one kind of input, such as text and images.

Prompt injection: untrusted content attempting to change instructions or trigger behavior.

RAG: retrieval-augmented generation; retrieve relevant material, then give it to a model as context.

Structured output: model output constrained to a declared data schema.

Token: a unit used to process and price model input and output; it is not exactly a word.

Tool or function call: a structured request from the model for application code to perform an operation.

Primary official sources

- [Gemini API documentation](#)
- [Interactions API overview](#)
- [Getting started](#)
- [Google GenAI SDK libraries](#)
- [Models](#)
- [Prompt design strategies](#)
- [Structured outputs](#)
- [Function calling](#)
- [File Search](#)
- [Grounding with Google Search](#)
- [Context caching](#)
- [Safety settings](#)
- [Safety and factuality guidance](#)
- [API keys](#)
- [Rate limits](#)
- [Deprecations](#)
- [Pricing](#)